9 优化计算

在前一章中，我们讨论了如何为高效的内存访问扫清道路。一旦完成这一点，就是时候看看 CPU 如何处理它从内存中获取的数据了。现代应用程序需要大量的 CPU 计算，尤其是那些涉及复杂图形、人工智能、加密货币挖矿和大数据处理的应用程序。在本章中，我们将专注于优化计算，这可以减少 CPU 需要完成的工作量，并提高程序的整体性能。

当应用 TMA 方法时，低效的计算通常反映在`Core Bound`类别中，以及在一定程度上反映在`Retiring`类别中。`Core Bound`类别代表了 CPU 乱序执行引擎内所有非内存问题引起的停顿。主要有两个类别：

- 软件指令之间的数据依赖性限制了性能。例如，一系列依赖操作可能导致指令级并行性（ILP）低下，浪费了许多执行槽。下一节将更详细地讨论数据依赖链。
- 硬件计算资源短缺（也称为执行吞吐量不足）。这表明某些执行单元过载（也称为执行端口竞争）。当工作负载频繁执行许多相同类型的指令时，可能会发生这种情况。例如，AI 算法通常执行大量的乘法运算，科学应用程序可能运行许多除法和平方根运算。但在任何给定的 CPU 核心中，乘法器和除法器的数量都是有限的。因此，当端口竞争发生时，指令排队等待执行。这种性能瓶颈非常特定于特定的 CPU 微架构，通常没有解决办法。

在 Section 6.1中，我们说高`Retiring`指标是代码表现良好的一个好指标。其背后的原因是执行没有停顿，CPU 以高速度退役指令。然而，有时它可能掩盖了真正的性能问题，即计算效率低下。一个工作负载可能执行了大量过于简单的指令，并没有做很多有用的工作。在这种情况下，高`Retiring`指标不会转化为高性能。

在本章中，我们将看看一些众所周知的技术，如函数内联、向量化和循环优化。这些代码转换旨在减少执行指令的总量，或者用更高效的指令替换它们。

9.1 数据流依赖

当程序语句引用前面语句的数据时，我们称这两个语句之间存在数据依赖性。有时人们也使用“依赖链”或“数据流依赖”等术语。我们最熟悉的例子如图 65 所示。要访问节点 `N+1`，我们应该首先取消引用指针 `N->next` 的引用。对于右边的循环，这是一个递归的数据依赖性，这意味着它跨越了循环的多个迭代。基本上，遍历一个链表是一个非常长的依赖链。

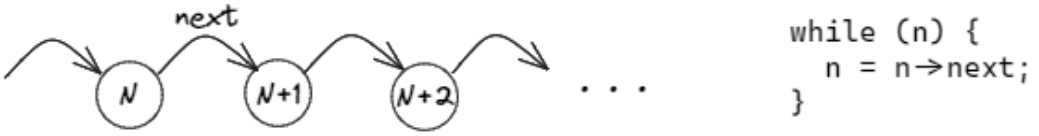


Figure 65: 遍历链表时的数据依赖性.

传统程序是在假设顺序执行模型的情况下编写的。在这个模型下，指令一个接一个地执行，原子地按照程序指定的顺序执行。然而，正如我们已经知道的，现代 CPU 不是这样构建的。它们被设计成乱序执行指令，并行执行，并以最大限度利用可用执行单元的方式执行。

当出现长数据依赖性时，处理器被迫以顺序执行代码，只利用了其全部能力的一部分。长依赖链阻碍了并行性，这违背了现代超标量 CPU 的主要优势。例如，指针追逐不能从 OOO 执行中获益，因此将以顺序 CPU 的速度运行。正如我们将在本节看到的那样，依赖链是性能瓶颈的主要来源。

您无法消除数据依赖性，它们是程序的基本属性。任何程序都接受输入来计算一些东西。事实上，人们已经开发了技术来发现语句之间的数据依赖性并构建数据流图。这称为依赖性分析，更适合编译器开发人员，而不是性能工程师。我们不希望为整个程序构建数据流图。相反，我们想在一段热代码（循环或函数）中找到一个关键的依赖链。

您可能会问：“如果不能摆脱依赖链，您可以做什么？”。嗯，有时这会成为性能的限制因素，不幸的是您将不得不忍受它。在本书的最后一章，我们将讨论硬件中打破依赖链的一种可能解决方案，称为值预测。现在，您应该寻找打破不必要的数据依赖链或使其执行重叠的方法。一个这样的例子显示在 Listing 9.1 中。与其他一些情况类似，我们在左边展示了源代码，右边是相应的 ARM 汇编代码。此外，这个代码示例包含在 Performance Ninja 的 Github 存储库中，因此您可以自己尝试一下。

这个小程序模拟了随机粒子运动。我们有 1000 个粒子在 2D 表面上运动，没有约束，这意味着它们可以离它们的起始位置尽可能远。每个粒子由其在 2D 表面上的 x 和 y 坐标以及速度定义。初始 x 和 y 坐标在范围 [-1000,1000] 内，速度在范围 [0;1] 内，不会改变。程序模拟每个粒子 1000 个运动步长。对于每个步骤，我们使用随机数生成器 (RNG) 生成一个角度，该角度设置粒子的运动方向。然后，我们相应地调整粒子的坐标。

考虑到手头的任务，您决定自己编写 RNG、正弦和余弦函数，以牺牲一些精度使其尽可能快。毕竟，这是随机运动，所以这是一个不错的权衡。您选择中等质量的 `XorShift` RNG，因为它只有 3 个移位和 3 个异或操作。还有什么更简单的？另外，您很快搜索了网络，使用多项式找到了正弦和余弦近似值，既准确又快速。

让我们快速检查一下生成的 ARM 汇编代码：

- 前三个 `eor` 指令与 `lsl` 或 `lsr` 组合对应于 `XorShift32::gen()` 函数。
- 接下来的 `ucvtf` 和 `fmul` 用于将角度从度转换为弧度（代码第 35 行）。
- 正弦和余弦函数都具有两个 `fmul` 和一个 `fmadd` 操作。余弦还具有附加的 `fadd`。
- 最后，我们再有一对 `fmadd` 来分别计算 x 和 y，以及 `stp` 指令将坐标对存储回原处。

使用 Clang-17 C++ 编译器编译了代码并在 Mac mini (Apple M1, 2020) 上运行。期望这段代码“飞起来”，但是有一个非常讨厌的性能问题使程序变慢。不提前阅读文本，你能在代码中找到一个递归依赖链吗?

代码清单: 二维表面上的随机粒子运动

```
1  struct Particle {
2    float x; float y; float velocity;
3  };
4
5  class XorShift32 {
6    uint32_t val;
7  public:
8    XorShift32 (uint32_t seed) : val(seed) {}
9    uint32_t gen() {
10     val ^= (val << 13);
11     val ^= (val >> 17);
12     val ^= (val << 5);
13     return val;
14   }
15 };
16
17 static float sine(float x) {
18   const float B = 4 / PI_F;
19   const float C = -4 / ( PI_F * PI_F);
20   return B * x + C * x * std::abs(x);
21 }
22 static float cosine(float x) {
23   return sine(x + (PI_F / 2));
```

```
.loop:
  eor    w0, w0, w0, lsl #13
  eor    w0, w0, w0, lsr #17
  eor    w0, w0, w0, lsl #5
  ucvtf  s1, w0
  fmov   s2, w9
  fmul   s2, s1, s2
  fmov   s3, w10
  fadd   s3, s2, s3
  fmov   s4, w11
  fmul   s5, s3, s3
```

```
}                                                       fmov   s6, w12
                                                        fmul   s5, s5, s6
/* Map degrees [0;UINT32_MAX) to radians [0;2*pi)*/     fmadd  s3, s3, s4, s5
float DEGREE_TO_RADIAN = (2 * PI_D) / UINT32_MAX;       ldp    s6, s4, [x1, #0x4]
                                                        ldr    s5, [x1]
void particleMotion(vector<Particle> &particles,        fmadd  s3, s3, s4, s5
                    uint32_t seed) {                    fmov   s5, w13
 XorShift32 rng(seed);                                  fmul   s5, s1, s5
 for (int i = 0; i < STEPS; i++)                        fmul   s2, s5, s2
  for (auto &p : particles) {                           fmadd  s1, s1, s0, s2
   uint32_t angle = rng.gen();                          fmadd  s1, s1, s4, s6
   float angle_rad = angle * DEGREE_TO_RADIAN;          stp    s3, s1, [x1], #0xc
   p.x += cosine(angle_rad) * p.velocity;               cmp    x1, x16
   p.y += sine(angle_rad) * p.velocity;                 b.ne   .loop
  }
}
```

恭喜您找到了它！存在一个关于 `XorShift32::val` 的循环依赖。要生成下一个随机数，生成器必须首先生成前一个数字。`gen()` 方法的下一个调用将基于前一个数字生成数字。图 66 直观地显示了有问题的循环进位依赖。请注意，计算粒子坐标的代码（将角度转换为弧度，正弦，余弦，乘以速度）会在相应的随机数准备好后立即开始执行，但不会更早。

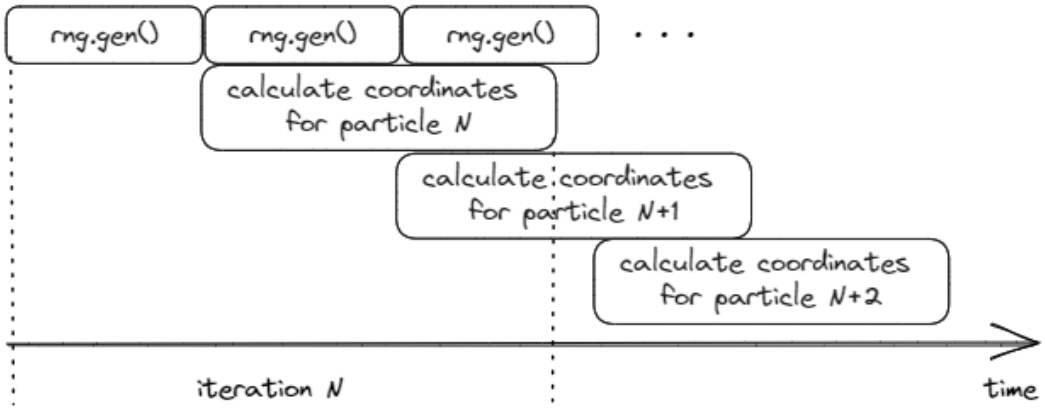


Figure 66: 在 Listing 9.1 中依赖执行的可视化

每个粒子坐标的计算代码彼此独立，因此向左拉伸它们以更多地重叠它们的执行可能会有益。您可能想知道：“但是这三个（或六个）指令如何拖慢整个循环？”。确实，循环中还有许多其他“繁重”指令，例如 `fmul` 和 `fmadd`。然而，它们不在关键路径上，因此可以与其他指令并行执行。并且由于现代 CPU 非常宽，它们将同时执行来自多个迭代的指令。这允许 OOO 引擎在循环的不同迭代中有效地找到并行性（独立指令）。

让我们做一些粗略的计算。[153] 每个 `eor` 和 `lsl` 指令的延迟为 2 个周期，一个周期用于移位，一个周期用于异或。我们有三个依赖的 `eor + lsl` 对，因此生成下一个随机数需要 6 个周期。这是我们这个循环的绝对最小值，我们不能以每迭代 6 个周期以上的速度运行。后续代码至少需要 20 个周期延迟才能完成所有 `fmul` 和 `fmadd` 指令。但这没关系，因为它们不在关键路径上。重要的是这些指令的吞吐量。经验法则：如果一条指令处于关键路径上，请查看其

[153] 苹果公司没有公布他们产品的指令延迟和吞吐量，但有一些实验可以说明这一点，这里有一个这样的研究:https://dougallj.github.io/applecpu/firestorm-simd.html。由于这是非官方的数据来源，你应该有所保留。

延迟；如果它不在关键路径上，请查看其吞吐量。在每个循环迭代中，我们有 5 个 `fmul` 和 4 个 `fmadd` 指令，它们都在同一组执行单元上执行。M1 处理器可以每周期运行 4 条这种类型的指令，因此发出所有 `fmul` 和 `fmadd` 指令至少需要 `9/4 = 2.25` 个周期。因此，我们有两个性能限制：第一个由软件强制（每个迭代 6 个周期，由于依赖链），第二个由硬件强制（每个迭代 2.25 个周期，由于执行单元的吞吐量）。现在我们受第一个限制约束，但我们可以尝试打破依赖链以接近第二个限制。

解决这个问题的一种方法是使用额外的 RNG 对象，使其一个为循环的偶数迭代提供数据，另一个为奇数迭代提供数据，如 Listing 153 所示。请注意，我们还手动展开循环。现在我们有两个独立的依赖链，可以并行执行。有人可能会说这改变了程序的功能，但用户无法分辨，因为粒子的运动是随机的。另一种解决方案是选择一个内部依赖链更少的 RNG。

代码清单: 二维表面上的随机粒子运动

```
void particleMotion(vector<Particle> &particles,
                    uint32_t seed1, uint32_t seed2) {
  XorShift32 rng1(seed1);
  XorShift32 rng2(seed2);
  for (int i = 0; i < STEPS; i++) {
    for (int j = 0; j + 1 < particles.size(); j += 2) {
      uint32_t angle1 = rng1.gen();
      float angle_rad1 = angle1 * DEGREE_TO_RADIAN;
      particles[j].x += cosine(angle_rad1) * particles[j].velocity;
      particles[j].y += sine(angle_rad1)   * particles[j].velocity;
      uint32_t angle2 = rng2.gen();
      float angle_rad2 = angle2 * DEGREE_TO_RADIAN;
      particles[j+1].x += cosine(angle_rad2) * particles[j+1].velocity;
      particles[j+1].y += sine(angle_rad2)   * particles[j+1].velocity;
    }
    // remainder (not shown)
  }
}
```

完成此转换后，编译器开始自动向量化循环主体，即它将两个链粘合在一起并使用 SIMD 指令并行处理它们。为了隔离打破依赖链的影响，我们禁用了编译器向量化。

为了测量改变的影响，我们运行了“之前”和“之后”版本，观察到运行时间从每个迭代 19 毫秒降至每个迭代 10 毫秒。这几乎是 2 倍的加速。IPC 也从 4.0 上升到 7.1。为了尽职尽责，我们还测量了其他指标，以确保性能不会因其他原因意外提升。在原始代码中，MPKI 为 0.01，BranchMispredRate 为 0.2%，这意味着程序最初没有遭受缓存未命中或分支预测错误。这里还有另一个数据点：在英特尔的 Alderlake 系统上运行相同的代码时，它显示了 74% 的 Retiring 和 24% 的 Core Bound，这证实了性能受计算限制。

通过一些额外的更改，您可以将此解决方案通用化，以拥有您想要的任意数量的依赖链。对于 M1 处理器，测量结果表明拥有 2 个依赖链足以非常接近硬件限制。拥有超过 2 个链路的性能提升可以忽略不计。然而，一个趋势是 CPU 变得越来越宽，即它们越来越能够并行运行多个依赖链。这意味着未来的处理器可以受益于拥有超过 2 个依赖链。与往常一样，您应该测量并找到您的代码将在其上运行的平台的最佳点。

有时仅打破依赖链还不够。想象一下，您没有一个简单的 RNG，而是一个非常复杂的加密算法，它长达 10’000 个指令。因此，现在我们不再是一个非常短的 6 个指令依赖链，而是有 10’000 个指令位于关键路径上。您立即进行上述相同的更改，期待获得 2 倍的加速。只会看到性能稍有提升。发生了什么?

这里的问题是 CPU 无法“看到”第二个依赖链开始执行它。回想一下第 3 章，预留站 (RS) 的容量不足以提前看到 10’000 条指令，因为它比这小得多。因此，CPU 将无法重叠执行两个依赖链。为了修复它，我们需要交错这两个依赖链。使用这种方法，您需要更改代码，以便 RNG 对象同时生成两个数字，函数 gen() 中的 每个 语句都重复并交错。即使编译器内联所有代码并且可以清楚地看到两个链，它也不会自动交错它们，因此您需要小心这一点。您在执行此操作时可能遇到的另一个限制是寄存器压力。并行运行多个依赖链需要保持更多状态，因此需要更多寄存器。如果您用完寄存器，编译器会将它们溢出到堆栈，从而减慢程序速度。

作为结束时的思考，我们想强调找到关键依赖链的重要性。这并不总是容易的，但要知道您的循环、函数或代码段的关键路径上是什么至关重要。否则，您可能会发现自己在修复次要问题，而这些问题几乎没有影响。

9.2 内联函数

如果你是那种经常查看汇编代码的开发人员，你可能见过 CALL、PUSH、POP 和 RET 指令。在 x86 指令集中，CALL 和 RET 指令用于调用和返回函数。PUSH 和 POP 指令用于将寄存器值保存到堆栈上并恢复它。

函数调用的微妙之处由调用约定描述，即如何传递参数和顺序，如何返回结果，调用的函数必须保留哪些寄存器以及工作在调用方和被调用方之间如何分配。基于调用约定，当调用者调用一个函数时，它期望在被调用者返回后一些寄存器将保持相同的值。因此，如果被调用者需要更改应保留的寄存器之一，它需要在返回给调用者之前保存（PUSH）和恢复（POP）它们。一系列 PUSH 指令称为序言，一系列 POP 指令称为尾声。

当一个函数很小的时候，调用函数的开销（序言和尾声）可能非常明显。通过将函数体内联到调用位置，可以消除这种开销。函数内联是将对函数 F 的调用替换为对实际参数专门化的 F 代码的过程。内联是最重要的编译器优化之一。不仅因为它消除了调用函数的开销，而且还使其他优化变得可能。这是因为当编译器内联一个函数时，编译器分析的范围会扩大到一个更大的代码块。然而，也有缺点：内联可能会增加代码大小和编译时间[154]。

许多编译器中函数内联的主要机制依赖于成本模型。例如，在 LLVM 编译器中，它基于为每个函数调用（调用点）计算成本。内联函数调用的成本基于该函数中指令的数量和类型。如果成本低于阈值，通常是固定的阈值，则会进行内联，但在某些情况下可以变化[155]。除了通用成本模型之外，还有许多启发式方法可以在某些情况下覆盖成本模型的决策。例如：

- 微小的函数（包装器）几乎总是被内联。
- 只有一个调用点的函数是内联的首选候选项。
- 大型函数通常不会被内联，因为它们会膨胀调用函数的代码。

此外，有些情况下内联会有问题：

- 递归函数不能内联到自身。
- 通过指针引用的函数可以内联到直接调用的地方，但是该函数必须保留在二进制文件中，即它不能完全内联和消除。对于具有外部链接的函数也是如此。

正如我们之前所说，编译器在决定是否内联函数时倾向于使用成本模型方法，这在实践中通常效果很好。一般来说，依靠编译器做出所有内联决策并在需要时进行调整是一个很好的策略。成本模型无法考虑到每种可能的情况，这为改进留下了空间。有时候编译器需要开发人员的特殊提示。一种找到程序中潜在内联候选项的方法是查看分析数据，特别是函数序言和尾声的热度。下面是一个函数剖面的示例，其中序言和尾声消耗了函数时间的~50%：

```
开销 | 函数 `foo` 的源代码和反汇编
  (%)   |
-----------------------------------------
   3.77 :  418be0:  push   r15      # 序言
```

[154] 参见文章：https://aras-p.info/blog/2017/10/09/Forced-Inlining-Might-Be-Slow/。

[155] 例如，1) 当函数声明带有内联提示时，2) 当存在函数的分析数据时，或者 3) 当编译器优化大小 (-Os) 而不是性能 (-O2) 时。

```
    4.62 :  418be2:  mov    r15d,0x64
    2.14 :  418be8:  push   r14
    1.34 :  418bea:  mov    r14,rsi
    3.43 :  418bed:  push   r13
    3.08 :  418bef:  mov    r13,rdi
    1.24 :  418bf2:  push   r12
    1.14 :  418bf4:  mov    r12,rcx
    3.08 :  418bf7:  push   rbp
    3.43 :  418bf8:  mov    rbp,rdx
    1.94 :  418bfb:  push   rbx
    0.50 :  418bfc:  sub    rsp,0x8
    ...
    #                              # 函数体
    ...
    4.17 :  418d43:  add    rsp,0x8  # 尾声
    3.67 :  418d47:  pop    rbx
    0.35 :  418d48:  pop    rbp
    0.94 :  418d49:  pop    r12
    4.72 :  418d4b:  pop    r13
    4.12 :  418d4d:  pop    r14
    0.00 :  418d4f:  pop    r15
    1.59 :  418d51:  ret
```

当你看到热的 PUSH 和 POP 指令时，这可能是一个很强的指示，即函数

序言和尾声的时间可能会被节省，如果我们内联函数的话。请注意，即使序言和尾声很热，也不一定意味着内联函数会有利可图。内联会触发许多不同的更改，因此很难预测结果。在强制编译器内联函数之前，请始终测量更改代码的性能。

对于 GCC 和 Clang 编译器，可以使用 C++11 的 `[[gnu::always_inline]]` 属性作为内联 `foo` 的提示，如下面的代码示例所示。对于较早的 C++ 标准，可以使用 `__attribute__((always_inline))`。对于 MSVC 编译器，可以使用 `__forceinline` 关键字。

```
[[gnu::always_inline]] int foo() {
    // foo body
}
```

9.3 循环优化

循环是几乎所有高性能程序的核心。由于循环代表了执行大量次的代码片段，因此它们是执行时间花费最多的部分。在这样一个关键代码段进行微小的更改可能会对程序的性能产生重大影响。这就是为什么仔细分析程序中热点循环的性能并了解改进它们的可能方法非常重要。

要有效地优化循环，关键是要了解性能瓶颈。一旦找到占用大部分时间的循环，就尝试确定限制其性能的因素。通常，它将是以下一种或多种情况：内存延迟、内存带宽或机器的计算能力。Roofline 性能模型（Section 5.6）是评估不同循环相对于硬件理论最大值的性能的一个良好起点。自上而下的微架构分析（Section 6.1）也可以成为有关瓶颈的另一个很好的信息来源。

在本节中，我们将研究针对上述瓶颈类型最著名的循环优化。我们首先讨论低级别的优化，这些优化只会在一个循环中移动代码。此类优化通常有助于提高循环内部计算的有效性。接下来，我们将研究重构循环的高级别优化，这些优化通常会影响多个循环。第二类优化通常旨在改进内存访问，消除内存带宽和内存延迟问题。请注意，这不是所有已知循环转换的完整列表。有关下面讨论的每个转换的更详细信息，读者可以参考 [Cooper & Torczon, 2012]。

编译器可以自动识别执行某些循环转换的机会。然而，有时需要开发人员干预才能达到所需的结果。在本节的第二部分，我们将分享一些有关如何发现循环优化机会的想法。了解对给定循环进行了哪些转换以及编译器未能进行哪些优化是成功性能调优的关键之一。最后，我们将考虑使用多面体框架优化循环的另一种方法。

9.3.1 低级优化

首先，我们将考虑一些简单的循环优化，这些优化会改变单个循环内的代码：循环不变代码移动（Loop Invariant Code Motion，LICM）、循环展开（Loop Unrolling）、循环强度降低（Loop Strength Reduction，LSR）和循环取消开关（Loop Unswitching）。这些优化通常有助于提高具有高算术强度循环的性能（见 Section 5.6），即当循环受 CPU 计算能力限制时。通常，编译器在执行这类转换方面做得很好；然而，仍然有一些情况可能需要开发者的支持。我们将在后续章节中讨论这些情况。

循环不变代码移动（LICM）：在循环中评估且从不改变的表达式称为循环不变量。由于它们的值在循环迭代中不会改变，我们可以将循环不变表达式移出循环。我们通过将结果存储在临时变量中，并在循环内部使用它来实现这一点（见 Listing 9.3.1）。现在所有不错的编译器在大多数情况下都能成功执行 LICM。

代码清单：循环不变代码移动

```
for (int i = 0; i < N; ++i)              for (int i = 0; i < N; ++i) {
  for (int j = 0; j < N; ++j)     =>       auto temp = c[i];
    a[j] = b[j] * c[i];                   for (int j = 0; j < N; ++j)
                                            a[j] = b[j] * temp;
                                         }
```

循环展开：循环中的累加变量是其值依赖于循环迭代次数的变量。例如，`v = f(i)`，其中`i`是迭代次数。在每次迭代中修改累加变量可能是不必要的且代价高昂。相反，我们可以展开循环，并为累加变量的每次增量执行多次迭代（见 Listing 9.3.1）。

代码清单：循环展开

```
for (int i = 0; i < N; ++i)              for (int i = 0; i+1 < N; i+=2) {
  a[i] = b[i] * c[i];           =>         a[i]   = b[i]   * c[i];
                                           a[i+1] = b[i+1] * c[i+1];
                                         }
```

循环展开的主要好处是每次迭代可以执行更多的计算。每次迭代结束时，索引值必须被增量、测试，并且如果还有更多迭代要处理，控制流会被分支回循环的顶部。这项工作可以被视为循环的“税”，可以减少。通过在 Listing 9.3.1中将循环展开 2 倍，我们将执行的比较和分支指令数量减少了一半。

循环展开是一种众所周知的优化；尽管如此，许多人对此感到困惑，并尝试手动展开循环。我建议没有开发者应该手动展开任何循环。首先，编译器在这方面做得很好，通常会非常优化地进行循环展开。第二个原因是，由于它们的乱序推测执行引擎（见 Chapter 3），处理器具有“嵌入式展开器”。当处理器等待第一次迭代中的长延迟指令完成时（例如加载、除法、微码指令、长依赖链），它会推测性地开始执行第二次迭代的指令，并且只等待循环携带的依赖项。这跨越了多个迭代，有效地在指令重排序缓冲区（ROB）中展开了循环。

循环强度降低（LSR）：用更便宜的指令替换昂贵的指令。这种转换可以应用于使用累加变量的所有表达式。强度降低通常应用于数组索引。编译器通过分析变量在循环迭代中的演变来执行 LSR。在 LLVM 中，它被称为标量演化

(Scalar Evolution，SCEV)。在 Listing 9.3.1中，编译器相对容易证明内存位置b[i*10]是循环迭代次数i的线性函数，因此它可以将昂贵的乘法替换为更便宜的加法。

代码清单：循环强度降低

```
for (int i = 0; i < N; ++i)                int j = 0;
  a[i] = b[i * 10] * c[i];       =>        for (int i = 0; i < N; ++i) {
                                             a[i] = b[j] * c[i];
                                             j += 10;
                                           }
```

循环取消开关（Loop Unswitching）：如果循环内部有一个条件语句，并且它是不变的，我们可以将其移出循环。我们通过复制循环体，并将其版本放置在条件语句的每个if和else子句中来实现这一点（见 Listing 9.3.1）。虽然循环取消开关可能会使编写的代码量翻倍，但这些新循环现在可以分别进行优化。

代码清单：循环取消开关

```
for (i = 0; i < N; i++) {                  if (c)
  a[i] += b[i];                              for (i = 0; i < N; i++) {
  if (c)                          =>           a[i] += b[i];
    b[i] = 0;                                  b[i] = 0;
}                                            }
                                           else
                                             for (i = 0; i < N; i++) {
                                               a[i] += b[i];
                                             }
```

9.3.2 高级优化

还有一类循环转换会改变循环的结构，通常会影响到多个嵌套循环。我们将探讨循环交换（Loop Interchange）、循环阻塞（Loop Blocking，也称为平铺或 Tiling）、以及循环融合和分布（Loop Fusion and Distribution，也称为 Fission）。这些转换的目的是改善内存访问，消除内存带宽和内存延迟的瓶颈。从编译器的角度来看，证明这类转换的合法性并证明它们的性能优势是非常困难的。在这个意义上，开发者处于一个更好的位置，因为他们只需要关心他们特定代码片段中转换的合法性，而不需要关心可能发生的每一种情况。不幸的是，这也意味着我们通常需要手动进行这样的转换。

循环交换（Loop Interchange）：交换嵌套循环的顺序的过程。内循环中使用的累加变量切换到外循环，反之亦然。Listing 9.3.2展示了交换i和j嵌套循环的例子。循环交换的主要目的是对多维数组的元素进行顺序内存访问。通过遵循元素在内存中布局的顺序，我们可以提高内存访问的空间局部性，使我们的代码更加友好于缓存。这种转换有助于消除内存带宽和内存延迟的瓶颈。

代码清单：循环交换

```
for (i = 0; i < N; i++)                    for (j = 0; j < N; j++)
  for (j = 0; j < N; j++)          =>      for (i = 0; i < N; i++)
    a[j][i] += b[j][i] * c[j][i];            a[j][i] += b[j][i] * c[j][i];
```

循环交换仅在循环是完全嵌套的情况下才合法。完全嵌套的循环是指所有语句都在最内层循环中。交换不完全嵌套的循环嵌套更困难，但仍然可能，可以在Codee[156]目录中查看一个例子。

[156] Codee: 完美循环嵌套 - [https://www.codee.com/catalog/glossary-perfect-loop-nesting/]](https://www.codee.com/catalog/glossary-perfect-loop-nesting/)

循环阻塞（Loop Blocking，Tiling）：这种转换的思想是将多维执行范围分割成更小的块（块或平铺），以便每个块都能适应 CPU 缓存。如果一个算法处理大型多维数组并对其元素进行跨步访问，那么缓存利用率很可能很低。每一次这样的访问可能会将未来访问请求的数据推出缓存（缓存驱逐）。通过将算法分割成更小的多维块，我们确保循环中使用的数据在被重用之前一直留在缓存中。

在 Listing 156所示的例子中，算法对数组a的元素进行行主序遍历，同时对数组b的元素进行列主序遍历。循环嵌套可以被分割成更小的块，以最大化数组b中元素的重用。

代码清单：循环阻塞

```
// linear traversal                    // traverse in 8*8 blocks
for (int i = 0; i < N; i++)            for (int ii = 0; ii < N; ii+=8)
  for (int j = 0; j < N; j++)    =>      for (int jj = 0; jj < N; jj+=8)
    a[i][j] += b[j][i];                    for (int i = ii; i < ii+8; i++)
                                             for (int j = jj; j < jj+8; j++)
                                               a[i][j] += b[j][i];
```

循环阻塞是优化通用矩阵乘法（GEMM）算法的广泛已知方法。它增强了内存访问的缓存重用，并改善了算法的内存带宽和内存延迟。

通常，工程师会针对每个 CPU 核心的私有缓存大小（Intel 和 AMD 的 L1 或 L2，Apple 的 L1）来优化平铺算法。然而，私有缓存的大小随着代际变化而变化，因此硬编码块大小会带来一系列挑战。作为一种替代解决方案，可以使用缓存无关[157]算法，其目标是在任何大小的缓存上都能合理地工作。

循环融合和分布（Loop Fusion and Distribution，Fission）：当它们迭代相同的范围且不引用彼此的数据时，可以合并分开的循环。一个循环融合的例子在 Listing 157中展示。相反的过程称为循环分布（Fission），即将循环分割成单独的循环。

代码清单：循环融合和分布

```
for (int i = 0; i < N; i++)            for (int i = 0; i < N; i++) {
  a[i].x = b[i].x;                       a[i].x = b[i].x;
                                =>       a[i].y = b[i].y;
for (int i = 0; i < N; i++)            }
  a[i].y = b[i].y;
```

循环融合有助于减少循环开销（类似于循环展开），因为两个循环可以使用相同的累加变量。此外，循环融合还可以帮助提高内存访问的时间局部性。在 Listing 157中，如果结构体的x和y成员恰好位于同一缓存行上，那么融合两个循环会更好，因为我们可以避免两次加载相同的缓存行。这将减少缓存占用并提高内存带宽利用率。

然而，循环融合并不总是提高性能。有时，将循环分成多个阶段，预先过滤数据，排序和重新组织等会更好。通过将大循环分布成多个较小的循环，我们限制了每个循环迭代所需的数据量，有效地提高了内存访问的时间局部性。这在高缓存竞争的情况下特别有帮助，这种情况通常发生在大循环中。循环分布还减少了寄存器压力，因为每个循环迭代中进行的操作更少。此外，将大循环分解成多个较小的循环可能对 CPU 前端的性能有益，因为指令缓存利用率更好。最后，当分布后，每个小循环可以由编译器进一步单独优化。

循环展开和融合（Loop Unroll and Jam）：要执行这种转换，首先需要展开外循环，然后将多个内循环融合在一起，如 Listing 157所示。这种转换增加了内循环的 ILP（指令级并行性），因为内循环中执行了更多的独立指令。在代码示例中，内循环是一个归约操作，它累积数组a和b元素之间的差异。当我们以 2 的因子展开和融合循环嵌套时，我

[157] 缓存遗忘算法 - https://en.wikipedia.org/wiki/Cache-oblivious_algorithm

们实际上同时执行了初始外循环的 2 次迭代。这通过拥有 2 个独立的累加器来强调，这打破了初始变体中diffs的依赖链。

代码清单：循环展开和融合

```
for (int i = 0; i < N; i++)                 for (int i = 0; i+1 < N; i+=2)
  for (int j = 0; j < M; j++)                 for (int j = 0; j < M; j++) {
    diffs += a[i][j] - b[i][j];     =>          diffs1 += a[i][j]   - b[i][j];
                                                diffs2 += a[i+1][j] - b[i+1][j];
                                              }
                                            diffs = diffs1 + diffs2;
```

循环展开和融合可以在没有跨迭代依赖的情况下执行外循环，换句话说，内循环的两次迭代可以并行执行。此外，如果内循环的内存访问在外部循环索引（本例中的i）上是跨步的，那么这种转换也是有意义的，否则其他转换可能更适用。当内循环的迭代次数较低时，例如小于 4，展开和融合特别有用。通过进行转换，我们将更多的独立操作打包到内循环中，从而增加了 ILP。

展开和融合转换有时对于外循环向量化非常有用，而在编写本文时，编译器无法自动执行。在内循环的迭代次数对编译器不可见的情况下，它仍然可以向量化原始内循环，希望它执行足够多的迭代以触发向量化代码（下一节将详细介绍向量化）。但如果迭代次数较低，程序将使用慢速的标量版本循环。一旦我们进行展开和融合，我们允许编译器以不同的方式向量化代码：现在将内循环中的独立指令“粘合”在一起（也称为 SLP 向量化）。

9.3.3 发现循环优化机会

正如我们在本节开头讨论的，编译器会承担优化循环的大部分工作。你可以依赖它们在循环代码中进行所有明显的改进，比如消除不必要的工作，执行各种窥孔优化等。有时，编译器足够聪明，可以默认生成循环的快速版本，而其他时候我们可能需要自己重写代码来帮助编译器。正如我们之前所说，从编译器的角度来看，合法且自动地执行循环转换是非常困难的。通常，当编译器无法证明转换的合法性时，它们必须保守处理。

考虑 Listing 9.3.3中的代码。编译器不能将表达式strlen(a)移出循环体。因此，循环在每次迭代时都会检查我们是否到达了字符串的末尾，这显然很慢。编译器不能提升调用的原因是，可能存在数组a和b的内存区域重叠的情况。在这种情况下，将strlen(a)移出循环体将是非法的。如果开发者确信内存区域不重叠，他们可以使用restrict关键字声明函数foo的两个参数，即char* __restrict__ a。

代码清单：不能将 strlen 移出循环

```
void foo(char* a, char* b) {
  for (int i = 0; i < strlen(a); ++i)
    b[i] = (a[i] == 'x') ? 'y' : 'n';
}
```

有时，编译器可以通过编译器优化备注（见 Section 5.8）告知我们转换失败。然而，在这种情况下，无论是 Clang 10 还是 GCC 10 都无法明确告诉我们表达式strlen(a)没有被提升出循环。找出这一点的唯一方法是根据应用程序的优化分析文件检查生成的汇编代码的热点部分。分析机器代码需要基本的汇编语言阅读能力，但这是一项非常有益的活动。

首先尝试获取容易实现的成果是一个合理的策略。开发者可以使用编译器优化报告或检查循环的机器代码来寻找简单的改进。有时，可以通过用户指令调整编译器转换。例如，当我们发现编译器将我们的循环展开 4 倍时，我们可以检查使用更高的展开因子是否会提高性能。大多数编译器支持#pragma unroll(8)，这将指示编译器使用用户指定的展开因子。还有其他控制特定转换的指令，如循环向量化、循环分布等。为了获取完整的用户指令列表，我们邀请用户查阅编译器手册。

接下来，开发者应该识别循环中的瓶颈，并根据硬件理论最大值评估性能。从 Roofline Performance Model (Section 5.6) 开始，它将揭示开发者应该尝试解决的瓶颈。循环的性能受到以下一个或多个因素的限制：内存延迟、内存带宽或机器的计算能力。一旦识别出循环的瓶颈，开发者可以尝试应用本节前面讨论的某种转换。

尽管对于特定计算问题的优化技术众所周知，循环优化仍然是随着经验而来的“黑艺术”。我们建议你依赖编译器，并在必要时手动转换代码。最重要的是，尽可能保持代码简单，如果性能提升微不足道，不要引入不合理的复杂变化。

9.3.4 循环优化框架

多年来，研究人员已经开发了确定循环转换合法性的技术，并能够自动转换循环。其中一项发明就是多面体框架[158]。GRAPHITE[159] 是最早集成到生产编译器中的多面体工具之一。GRAPHITE 基于从 GCC 的低级中间表示 GIMPLE 提取的多面体信息，执行一系列经典的循环优化。GRAPHITE 证明了这种方法的可行性。

后来，LLVM 编译器开发了自己的多面体框架，称为Polly[160]。Polly 是 LLVM 的一个高级循环和数据局部性优化基础设施。它使用基于整数多面体的抽象数学表示来分析和优化程序的内存访问模式。Polly 执行经典的循环转换，特别是平铺和循环融合，以提高数据局部性。这个框架在许多知名基准测试上显示出显著的加速效果 [Grosser et al., 2012]。以下是一个示例，展示了 Polly 如何将Polybench 2.0[161] 基准测试套件中的通用矩阵乘法（GEMM）核心的执行速度提高近 30 倍：

```
$ clang -O3 gemm.c -o gemm.clang
$ time ./gemm.clang
real 0m6.574s
$ clang -O3 gemm.c -o gemm.polly -mllvm -polly
$ time ./gemm.polly
real 0m0.227s
```

Polly 是一个强大的循环优化框架；然而，它仍然错过了一些常见且重要的情况。[162] 它在 LLVM 基础设施的标准优化流程中未启用，需要用户提供明确的编译器选项来使用它（`-mllvm -polly`）。使用多面体框架是在寻找加速循环的方法时的一个可行选择。

9.3.5 向量化

在现代处理器上，使用 SIMD 指令可以在常规未向量化（标量）代码上实现巨大的加速。在进行性能分析时，软件工程师的首要任务之一是确保代码的热点部分被向量化。本节将指导工程师发现向量化的机会。对于现代 CPU 的 SIMD 能力的回顾，读者可以查看 Section 3.4。

通常，向量化会自动发生，无需用户干预，这称为自动向量化。在这种情况下，编译器会自动识别从源代码生成 SIMD 机器代码的机会。自动向量化可能是一个方便的解决方案，因为现代编译器为各种程序生成快速的向量化代码。

然而，在某些情况下，如果没有软件工程师的干预，自动向量化就不会成功，这可能是基于他们从编译器或分析数据中得到的反馈[163]。在这些情况下，程序员需要告诉编译器某个特定的代码区域是可向量化的，或者向量化是有利可图的。现代编译器有扩展功能，允许高级用户控制自动向量化过程，并确保代码的某些部分被有效地向量化。然而，这种控制是有限的。在后续章节中，我们将提供几个使用编译器提示的例子。

[158] 多面体框架 - https://en.wikipedia.org/wiki/Loop_optimization#The_polyhedral_or_constraint-based_framework.

[159] GRAPHITE 多面体框架 - https://gcc.gnu.org/wiki/Graphite.

[160] Polly - https://polly.llvm.org/.

[161] Polybench - https://web.cse.ohio-state.edu/~pouchet.2/software/polybench/.

[162] Why not Polly? - https://sites.google.com/site/parallelizationforllvm/why-not-polly.

[163] 例如，编译器优化报告，参见 Section 5.8。

需要注意的是，有一系列问题在其中 SIMD 很重要，而自动向量化就是不起作用，而且在未来也不太可能起作用。可以在 [Muła & Lemire, 2019] 中找到一个例子。编译器目前不会尝试外循环向量化。它们不太可能向量化浮点代码，因为结果在数值上会有所不同。涉及跨向量通道的排列或洗牌的代码也不太可能自动向量化，这可能仍然是编译器的一个难题。

自动向量化还有一个微妙的问题。随着编译器的发展，它们所做的优化在变化。在前一个编译器版本中成功自动向量化的代码可能在下一个版本中停止工作，反之亦然。此外，在代码维护或重构期间，代码的结构可能会发生变化，以至于自动向量化突然开始失败。这可能发生在原始软件编写很久之后，因此在这一点上修复或重新实现将更加昂贵。

当绝对需要生成特定的汇编指令时，不应依赖编译器的自动向量化。在这些情况下，可以使用编译器内联函数编写代码，我们将在 Section 9.4中讨论。在大多数情况下，内联函数提供了与汇编指令的 1 对 1 映射。内联函数比内联汇编更容易使用，因为编译器负责寄存器分配，并且它们允许程序员在代码生成上保留相当大的控制权。然而，它们仍然通常是冗长且难以阅读的，并且可能受到各种编译器的行为差异甚至错误的影响。

在低成本但不可预测的自动向量化和冗长/不可读但可预测的内在函数之间，可以使用围绕内在函数的包装库。这些库往往更易读，可以将编译器修复集中在库中，而不是分散在用户代码中，并且仍然允许开发人员控制生成的代码。许多这样的库存在，它们支持最新或“奇特 (exotic)”操作的范围以及它们支持的平台数量各不相同。据我们所知，Highway 是目前唯一完全支持可扩展向量（如 SVE 和 RISC-V V 指令集）的库。需要注意的是，本文作者之一是该库的技术负责人。它将在 Section 9.4 中介绍。

请注意，使用内在函数或包装库时，仍然建议使用 C++ 编写初始实现。这样可以通过将原始代码的结果与新的向量化实现进行比较，从而快速原型化和验证正确性。

在本节的其余部分，我们将讨论几种方法，尤其是内循环向量化，因为它是自动向量化最常见的类型。其他两种类型，外循环向量化和 SLP (Superword-Level Parallelism) 向量化，将在附录 B 中提到。

9.3.6 编译器自动向量化

多重障碍可能会阻止自动向量化，其中一些障碍是编程语言语义固有的。例如，编译器必须假设无符号循环索引可能会溢出，这可能会阻止某些循环转换。另一个例子是 C 编程语言所做的假设：程序中的指针可能指向重叠的内存区域，这可能会使程序分析变得非常困难。另一个主要障碍是处理器本身的设计。在某些情况下，处理器没有针对某些操作的高效向量指令。例如，大多数处理器上没有可用的受控（位掩码控制）加载和存储操作。另一个例子是不同大小向量寄存器之间进行有符号整数到双精度浮点数的向量宽格式转换。尽管存在所有这些挑战，软件开发者仍然可以解决许多挑战并启用向量化。在后面的部分中，我们将提供如何与编译器合作的指导，确保热点代码被编译器向量化。

向量化器通常分为三个阶段：合法性检查、盈利性检查和转换本身：

- **合法性检查 (Legality-check)**：在这个阶段，编译器检查是否可以合法地将循环（或其他类型的代码区域）转换为使用向量。循环向量化器检查循环的迭代是否连续，这意味着循环线性进展。向量化器还确保循环中的所有内存和算术操作可以扩展为连续操作。循环的控制流在所有通道中是统一的，内存访问模式也是统一的。编译器必须以某种方式检查或确保生成的代码不会触及它不应该触及的内存，并且操作的顺序将被保留。编译器需要分析指针的可能范围，如果有任何缺失信息，它必须假设转换是非法的。合法性阶段收集了一系列要求，这些要求需要发生以便向量化循环合法。
- **盈利性检查 (Profitability-check)**：接下来，向量化器检查转换是否盈利。它比较不同的向量化因素，并找出哪个向量化因素执行最快。向量化器使用成本模型预测不同操作的成本，例如标量加法或向量加载。它需要考虑将数据洗牌到寄存器中添加的指令，预测寄存器压力，并估计确保允许向量化的前提条件得到满足的循环守卫的成本。检查盈利性的算法很简单：1) 将代码中所有操作的成本加起来，2) 比较每种代码版本的成本，3)

将成本除以预期的执行次数。例如，如果标量代码的成本为 8 个周期，向量化代码的成本为 12 个周期，但一次执行 4 次循环迭代，那么向量化版本的循环可能更快。

- **转换 (Transformation)**：最后，在向量化器确定转换是合法和盈利之后，它转换代码。这个过程还包括插入使向量化成为可能的守卫。例如，大多数循环使用未知的迭代次数，所以编译器必须生成循环的标量版本，除了向量化版本的循环之外，以处理最后几次迭代。编译器还必须检查指针是否重叠等。所有这些转换都是在合法性检查阶段收集的信息的基础上完成的。

9.3.7 发现向量化机会

阿姆达尔定律[164] 告诉我们，我们应该只分析程序执行过程中使用最多的代码部分。因此，性能工程师应该专注于由分析工具突出显示的代码热点。如前所述，向量化最常应用于循环。

发现提高向量化的机会应该从分析程序中的热点循环开始，并检查编译器执行了哪些优化。查看编译器向量化备注（见 Section 5.8）是了解这一点的最简单方法。现代编译器可以报告某个循环是否被向量化，并提供额外的细节，例如向量化因子（VF）。在编译器无法向量化循环的情况下，它也能够说明失败的原因。

使用编译器优化报告的另一种方法是检查汇编输出。最好使用一个分析工具的输出，该工具显示给定循环的源代码与生成的汇编指令之间的对应关系。这样，你只关注重要的代码，即热点代码。然而，理解汇编语言比理解像 C++ 这样的高级语言要困难得多。可能需要一些时间来弄清楚编译器生成的指令的语义。但这项技能非常有价值，通常能提供有价值的见解。有经验的开发者可以通过查看指令助记符和这些指令使用的寄存器名称，快速判断代码是否被向量化。例如，在 x86 ISA 中，向量指令操作打包数据（因此在它们的名称中有`P`），并使用`XMM`、`YMM`或`ZMM`寄存器，例如`VMULPS XMM1, XMM2, XMM3`将`XMM2`和`XMM3`中的四个单精度浮点数相乘，并将结果保存在`XMM1`中。但要小心，人们经常从看到使用`XMM`寄存器得出结论，认为这是向量代码——这不一定正确。例如，`VMULSS`指令只会乘以一个单精度浮点值，而不是四个。

开发者在尝试加速可向量化代码时经常遇到的一些常见情况如下。下面我们介绍四种典型场景，并给出每种情况下的一般指导。

9.3.7.1 向量化是非法的

在某些情况下，遍历数组元素的代码根本不可向量化。向量化备注非常有效地解释了出了什么问题以及为什么编译器无法向量化代码。Listing 165 显示了一个循环内部的依赖关系，该依赖关系阻止了向量化[165]。

代码清单: 向量化: 读写后写依赖。

```
void vectorDependence(int *A, int n) {
  for (int i = 1; i < n; i++)
    A[i] = A[i-1] * 2;
}
```

一些循环由于上述硬件限制无法向量化，但另一些循环在放松特定约束时可以向量化。有时，编译器无法向量化循环仅仅是因为它无法证明这样做是合法的。编译器通常非常保守，只会确信不会破坏代码时才进行转换。此类软限制可以通过向编译器提供额外的提示来放松。

例如，当转换执行浮点运算的代码时，向量化可能会改变程序的行为。浮点加法和乘法是交换的，这意味着您可以交换左手侧和右手侧而不改变结果：`(a + b == b + a)`。但是，这些操作不是关联的，因为舍入 (rounding) 发生在不同的时间：`((a + b) + c) != (a + (b + c))`。Listing 165 中的代码无法由编译器自动向量化。原因是向量化会将变量 `sum` 更改为向量累加器，这将改变运算顺序，并可能导致不同的舍入 (rounding) 决策和不同的结果。

[164] 阿姆达尔定律 - https://en.wikipedia.org/wiki/Amdahl's_law。

[165] 一旦展开几个循环迭代，很容易发现读后写依赖关系。请参见 Section 5.8 中的示例。

代码清单: 向量化: 浮点运算。

```
// a.cpp
float calcSum(float* a, unsigned N) {
  float sum = 0.0f;
  for (unsigned i = 0; i < N; i++) {
    sum += a[i];
  }
  return sum;
}
```

然而，如果程序可以容忍最终结果有一点小的不准确（通常情况下是这样），我们可以将此信息传达给编译器以启用向量化。Clang 和 GCC 编译器都有一个标志 `-ffast-math`[166]，它允许这种转换：

```
$ clang++ -c a.cpp -O3 -march=core-avx2 -Rpass-analysis=.*
...
a.cpp:5:9: remark: loop not vectorized: cannot prove it is safe to reorder floating-point
    operations; allow reordering by specifying '#pragma clang loop vectorize(enable)' before the
    loop or by providing the compiler option '-ffast-math'. [-Rpass-analysis=loop-vectorize]
...
$ clang++ -c a.cpp -O3 -ffast-math -Rpass=.*
...
a.cpp:4:3: remark: vectorized loop (vectorization width: 4, interleaved count: 2)
    [-Rpass=loop-vectorize]
...
```

不幸的是，此标志涉及微妙且潜在危险的行为变化，包括非数字 (NaN)、带符号零、无穷大和次正规数。由于第三方代码可能还没有准备好应对这些影响，因此不应在不仔细验证结果（包括边缘情况）的情况下在大段代码中启用此标志。

让我们看另一个典型情况，编译器可能需要开发人员的支持才能执行向量化。当编译器无法证明循环操作的是具有非重叠内存区域的数组时，它们通常会选择更安全的选项。让我们重新审视 Section 5.8 中 Listing 5.8 提供的示例。当编译器尝试向量化 Listing 166 中呈现的代码时，它通常无法做到这一点，因为数组 `a`、`b` 和 `c` 的内存区域可能重叠。

代码清单: a.c

```
void foo(float* a, float* b, float* c, unsigned N) {
  for (unsigned i = 1; i < N; i++) {
    c[i] = b[i];
    a[i] = c[i-1];
  }
}
```

这是由 GCC 10.2 提供的优化报告（使用 `-fopt-info` 启用）：

```
$ gcc -O3 -march=core-avx2 -fopt-info
a.cpp:2:26: optimized: loop vectorized using 32 byte vectors
a.cpp:2:26: optimized: loop versioned for vectorization because of possible aliasing
```

[166] 编译器标志 `-Ofast` 启用 `-ffast-math` 以及 `-O3` 编译模式。

GCC 识别到数组 a、b 和 c 的内存区域可能存在重叠，并创建了相同循环的多个版本。编译器插入了运行时检查[167]来检测内存区域是否重叠。基于这些检查，它会在向量化和标量[168]版本之间进行调度。在这种情况下，向量化会带来插入可能代价高昂的运行时检查的成本。

如果开发人员知道数组 a、b 和 c 的内存区域不会重叠，可以在循环前面插入 `#pragma GCC ivdep`[169] 或使用 `__restrict__` 关键字，如 Listing 5.8 所示。此类编译器提示将消除 GCC 编译器插入上述运行时检查的需要。

编译器本质上是静态工具：它们只根据所使用的代码进行推理。例如，一些动态工具（例如 Intel Advisor）可以检测跨迭代依赖或访问具有重叠内存区域的数组等问题是否确实出现在给定的循环中。但要记住，这类工具只提供建议。不加思索地插入编译器提示可能会导致实际问题。

9.3.7.2 **向量化无益** 在某些情况下，编译器可以向量化循环，但认为这样做没有好处。在 Listing 9.3.7.2 中呈现的代码中，编译器可以向量化对数组 A 的内存访问，但需要将对数组 B 的访问拆分为多个标量加载。这种分散/收集模式相对昂贵，并且能够模拟操作成本的编译器经常会决定避免向量化具有这种模式的代码。

代码清单: 向量化: 没有好处。

```
// a.cpp
void stridedLoads(int *A, int *B, int n) {
  for (int i = 0; i < n; i++)
    A[i] += B[i * 3];
}
```

下面是 Listing 9.3.7.2中的代码的编译器优化报告:

```
$ clang -c -O3 -march=core-avx2 a.cpp -Rpass-missed=loop-vectorize
a.cpp:3:3: remark: the cost-model indicates that vectorization is not beneficial
    [-Rpass-missed=loop-vectorize]
  for (int i = 0; i < n; i++)
  ^
```

用户可以使用 `#pragma` 提示强制 Clang 编译器向量化循环，如 Listing 170 所示。但是，请记住，向量化是否真正有利很大程度上取决于运行时数据，例如循环的迭代次数。编译器无法获得这些信息，[170] 因此它们往往保守行事。开发人员可以在寻找性能提升空间时使用此类提示。

代码清单: 向量化: 没有好处。

```
// a.cpp
void stridedLoads(int *A, int *B, int n) {
#pragma clang loop vectorize(enable)
  for (int i = 0; i < n; i++)
    A[i] += B[i * 3];
}
```

开发人员应注意使用向量化代码的隐藏成本。使用 AVX 和特别是 AVX-512 向量指令可能会导致频率降频或启动开销，在某些 CPU 上还会影响后续代码数微秒。向量化部分的代码应该足够热门，以证明使用 AVX-512 的合理性。[171]

[167] 请参阅 easyperf 博客上的示例: https://easyperf.net/blog/2017/11/03/Multiversioning_by_DD.
[168] 但循环的标量版本仍然可以展开。
[169] 它是 GCC 特定的编译器指令。对于其他编译器，请查阅相应的手册。
[170] 除了优化分析文件引导优化（参见 Section 11.7）。
[171] 有关更多细节，请阅读这篇博客文章: https://travisdowns.github.io/blog/2020/01/17/avxfreq1.html.

例如，排序 80 KiB 的数据被发现足以摊销这种开销，使向量化变得有价值。[172]

9.3.7.3 循环已向量化，但使用标量版本 在某些情况下，编译器可以成功地向量化代码，但向量化代码不会显示在性能分析器中。检查循环的对应汇编时，通常很容易找到循环体的向量化版本，因为它使用了向量寄存器（程序其他部分不常用），并且代码是展开的，并填充了检查和多个版本以启用不同的边缘情况。

如果生成的代码没有执行，一个可能的原因是编译器生成的代码假设循环执行次数高于程序使用的次数。例如，要在现代 CPU 上高效地进行向量化，程序员需要向量化并利用 AVX2，并且还要将循环展开 4-5 次，为流水线化的 FMA 单元生成足够的工作。这意味着每次循环迭代都需要处理大约 40 个元素。许多循环可能运行的循环执行次数低于这个值，并可能退回到使用标量剩余循环。很容易检测这些情况，因为标量剩余循环会在性能分析器中亮起，而向量化代码将保持冷状态。

针对这个问题的解决方案是强制向量化器使用较低的向量化因子或展开计数，以减少循环处理的元素数量，并使具有较低迭代次数的更多循环, 能够访问快速的向量化循环体。开发人员可以使用 `#pragma` 提示来实现这一点。对于 Clang 编译器，可以使用 `#pragma clang loop vectorize_width(N)`，如 easyperf 博客上的文章 [173] 所示。

9.3.7.4 循环以次优方式向量化 当您看到一个循环被向量化并在运行时执行时，该部分程序可能已经运行良好。但是，也有一些例外情况。有时，人类专家可以编写出性能优于编译器生成的代码。

由于以下几个因素，最佳向量化因子可能并不直观。首先，人类很难在脑海中模拟 CPU 的操作，除了尝试多种配置之外别无其他方法。涉及多个向量通道的向量洗牌可能比预期的更昂贵或更便宜，这取决于许多因素。其次，在运行时，程序可能会表现出不可预测的行为，这取决于端口压力和许多其他因素。这里的建议是尝试强制向量器选择一个特定的向量化因子和展开因子，并测量结果。向量化编译器指令可以帮助用户枚举不同的向量化因子并找出性能最佳的因子。每个循环可能的配置相对较少，并且在典型输入上运行循环是人类可以做到的，而编译器却做不到。

最后，有时循环的标量非向量化版本比向量化版本性能更好。这可能是因为使用了昂贵的向量操作，例如“gather/scatter”加载、掩码、洗牌等，编译器必须使用这些操作才能进行向量化。性能工程师也可以尝试通过不同的方式禁用向量化。对于 Clang 编译器，可以通过编译器选项 `-fno-vectorize` 和 `-fno-slp-vectorize` 或针对特定循环的提示（例如，`#pragma clang loop vectorize(enable)`）来实现。

9.3.7.5 使用显式向量化的语言 向量化还可以通过用专用于并行计算的编程语言重写程序的部分来实现。这些语言使用特殊的结构和对程序数据的了解，将代码高效地编译成并行程序。最初，这些语言主要用于将工作卸载到特定处理单元，例如图形处理单元 (GPU)、数字信号处理器 (DSP) 或现场可编程门阵列 (FPGA)。然而，其中一些编程模型也能够针对您的 CPU（例如 OpenCL 和 OpenMP）。

其中一种这样的并行语言是 Intel® Implicit SPMD 程序编译器 (ISPC)(https://ispc.github.io/)[174]，我们将在本节稍作介绍。ISPC 语言基于 C 编程语言，并使用 LLVM 编译器基础架构为许多不同的架构生成优化代码。ISPC 的关键特性是“接近金属”的编程模型和跨 SIMD 架构的性能可移植性。它要求从编写程序的传统思维方式转变，但为程序员提供了更多控制 CPU 资源利用率的手段。

ISPC 的另一个优点是其互操作性和易用性。ISPC 编译器生成标准对象文件，可以与传统 C/C++ 编译器生成的代码链接。ISPC 代码可以很容易地插入任何原生项目，因为用 ISPC 编写的函数可以像 C 代码一样调用。

Listing 174 显示了我们之前在 Listing 165 中介绍的一个简单函数，用 ISPC 重写。ISPC 考虑程序将在并行实例中运行，基于目标指令集。例如，当使用 SSE 与 `float` 一起使用时，它可以并行计算 4 个操作。每个程序实例将在 `i` 的

[172] AVX-512 降频研究: 在 VQSort readme: https://github.com/google/highway/blob/master/hwy/contrib/sort/README.md#study-of-avx-512-downclocking 中

[173] 使用 Clang 的优化编译器指令 - https://easyperf.net/blog/2017/11/09/Multiversioning_by_trip_counts

[174] ISPC 编译器: https://ispc.github.io/.

向量值上操作，分别是 (0,1,2,3)、然后是 (4,5,6,7)，以此类推，一次有效地计算 4 个和。正如您所看到的，使用了一些非典型 C 和 C++ 的关键字：

- export 关键字意味着该函数可以从 C 兼容的语言调用。
- uniform 关键字意味着变量在程序实例之间共享。
- varying 关键字意味着每个程序实例都有自己的局部变量副本。
- foreach 与经典的 for 循环相同，除了它会将工作分配到不同的程序实例。

代码清单:ISPC 版本的数组元素求和。

```
export uniform float calcSum(const uniform float array[],
                             uniform ptrdiff_t count)
{
    varying float sum = 0;
    foreach (i = 0 ... count)
        sum += array[i];
    return reduce_add(sum);
}
```

由于函数 calcSum 必须返回单个值（一个 uniform 变量），而我们的 sum 变量是一个 varying，因此我们需要使用 reduce_add 函数来 收集 每个程序实例的值。ISPC 还负责生成剥离和剩余循环，以考虑未正确对齐或不是向量宽度倍数的数据。

“接近硬件”编程模型：传统 C 和 C++ 语言的一个问题是编译器并不总是向量化代码的关键部分。通常情况下，程序员会使用编译器内部函数（参见 Section 9.4），这绕过了编译器自动向量化，但通常很困难，并且需要在出现新指令集时进行更新。ISPC 通过默认假设每个操作都是 SIMD 来帮助解决这个问题。例如，ISPC 语句 sum += array[i] 被隐含地认为是一个 SIMD 操作，可以并行进行多次加法运算。ISPC 不是一个自动向量化编译器，它不会自动发现向量化机会。由于 ISPC 语言与 C 非常相似，它比使用内部函数好得多，因为它可以让您专注于算法而不是低级指令。此外，据报道，它在性能方面与手写的内部函数代码相匹配或优于手写的内部函数代码[175]。

性能可移植性：ISPC 可以自动检测您 CPU 的特性，以充分利用所有可用资源。程序员可以编写一次 ISPC 代码，并编译到许多向量指令集，例如 SSE4、AVX 和 AVX2。ISPC 还可以为不同的架构（如 x86 CPU、ARM NEON）生成代码，并具有实验性的 GPU 卸载支持。

9.4 编译器内联函数

有些类型的应用程序有值得大力调整的热点。然而，编译器在这些热点生成的代码并不总是符合我们的期望。例如，程序在循环中进行一些计算，而编译器以次优的方式向量化。这通常涉及到一些棘手或特殊的算法，我们可以为这些算法设计出更好的指令序列。使用 C 和 C++ 语言的标准结构让编译器生成所需的汇编代码可能非常困难，甚至是不可能的。

幸运的是，有可能在不编写低级汇编语言的情况下，强制编译器生成特定的汇编指令。为了实现这一点，可以使用编译器内联函数，它们反过来被翻译成特定的汇编指令。内联函数提供了与使用内联汇编相同的好处，但同时也提高了代码的可读性，允许编译器类型检查，协助指令调度，并有助于减少调试。在 Listing 9.4中的示例展示了如何通过编译器内联函数（函数bar）来编码函数foo中的相同循环。

代码清单: 编译器内部函数

[175] 使用 SIMD 内部函数的虚幻引擎的一些部分使用 ISPC 重写，从而获得了速度提升: https://software.intel.com/content/www/us/en/develop/articles/unreal-engines-new-chaos-physics-system-screams-with-in-depth-intel-cpu-optimizations.html.

```
void foo(float *a, float *b, float *c, unsigned N) {
  for (unsigned i = 0; i < N; i++)
    c[i] = a[i] + b[i];
}

#include <xmmintrin.h>

void bar(float *a, float *b, float *c, unsigned N) {
  __m128 rA, rB, rC;
  int i = 0;
  for (; i + 3 < N; i += 4){
    rA = _mm_load_ps(&a[i]);
    rB = _mm_load_ps(&b[i]);
    rC = _mm_add_ps(rA,rB);
    _mm_store_ps(&c[i], rC);
  }
  for (; i < N; i++) // remainder
    c[i] = a[i] + b[i];
}
```

当编译为 SSE 目标时，`foo` 和 `bar` 都将生成类似的汇编指令。但是，有几个注意事项。首先，当依赖自动矢量化时，编译器会插入所有必要的运行时检查。例如，它将确保有足够的元素来填充向量执行单元。其次，函数 `foo` 将有一个处理循环剩余部分的标量版本作为备用。最后，大多数向量内部函数假设数据是对齐的，因此为 `bar` 生成 `movaps`（对齐加载），而为 `foo` 生成 `movups`（未对齐加载）。考虑到这一点，开发人员使用编译器内部函数时必须自己注意安全方面。

使用非可移植的平台特定内部函数编写代码时，开发人员还应该为其他架构提供备选项。有关英特尔平台的所有可用内部函数列表，请参见此 参考: https://software.intel.com/sites/landingpage/IntrinsicsGuide/[176]。

9.4.1 内部函数的包装库

ISPC 的一次编写，多目标运行模式很有吸引力。然而，我们可能希望更紧密地集成到 C++ 程序中，例如与模板互操作，或者避免单独的构建步骤并使用相同的编译器。相反，内部函数提供更多的控制，但开发成本更高。

我们可以结合两者的优势并避免这些缺点，使用所谓的嵌入式领域特定语言，其中向量操作表示为普通的 C++ 函数。您可以将这些函数视为“可移植的内部函数 (portable intrinsics)”，例如 `Add` 或 `LoadU`。即使多次编译您的代码（每个指令集一次），也可以在普通 C++ 库中完成，方法是使用预处理器在不同的编译器设置下“重复”您的代码，但位于独特的命名空间中。一个例子是之前提到的 Highway 库，[177] 它只要求 C++11 标准。

与 ISPC 一样，Highway 也支持检测最佳可用指令集，这些指令集分为“簇 (clusters)”，在 x86 上对应于 Intel Core (S-SSE3)、Nehalem (SSE4.2)、Haswell (AVX2)、Skylake (AVX-512) 或 Icelake/Zen4 (AVX-512 with extensions)。然后它从相应命名空间调用您的代码。

与内部函数不同，代码保持可读性（每个函数没有前缀/后缀）和可移植性。

代码清单: 对数组元素求和的高速版本。

```
#include <hwy/highway.h>
```

[176] 英特尔内部函数指南 - https://software.intel.com/sites/landingpage/IntrinsicsGuide/。

[177] Highway 库: https://github.com/google/highway

```
float calcSum(const float* HWY_RESTRICT array, size_t count) {
  const ScalableTag<float> d;  // type descriptor; no actual data
  auto sum = Zero(d);
  size_t i = 0;
  for (; i + Lanes(d) <= count; i += Lanes(d)) {
    sum = Add(sum, LoadU(d, array + i));
  }
  sum = Add(sum, MaskedLoad(FirstN(d, count - i), d, array + i));
  return ReduceSum(d, sum);
}
```

注意循环处理向量大小Lanes(d)的倍数后余数的显式处理。虽然这更加冗长，但它使实际发生的事情变得可见，并允许进行优化，例如重叠最后一个向量而不是依赖于MaskedLoad，或者当count已知是向量大小时完全跳过余数。

Highway 支持超过 200 种操作，可以分为以下几类：

- 初始化 (Initialization)
- 获取/设置车道 (Getting/setting lanes)
- 获取/设置块 (Getting/setting blocks)
- 打印 (Printing)
- 元组 (Tuples)
- 算术 (Arithmetic)
- 逻辑 (Logical)
- 掩码 (Masks)
- 比较 (Comparisons)
- 内存 (Memory)
- 缓存控制 (Cache control)
- 类型转换 (Type conversion)
- 组合 (Combine)
- 洗牌/排列 (Swizzle/permute)
- 在 128 位块内进行洗牌 (Swizzling within 128-bit blocks)
- 归约 (Reductions)
- 加密 (Crypto)

有关操作的完整列表，请参见其文档[178]和常见问题解答。您还可以在在线Compiler Explorer中尝试它。

其他库包括 Eigen、nsimd、SIMDe、VCL 和 xsimd。请注意，从 Vc 库开始的 C++ 标准化工作导致了 std::experimental::simd，但它提供了非常有限的操作集，并且截至本文撰写时仅在 GCC 11 编译器上支持。

问题和练习

1. 使用本章讨论的技术解决以下实验作业：
   - `perf-ninja::function_inlining_1`
   - `perf-ninja::vectorization` 1 & 2
   - `perf-ninja::dep_chains` 1 & 2

[178] Highway 快速参考 - https://github.com/google/highway/blob/master/g3doc/quick_reference.md

- `perf-ninja::compiler_intrinsics` 1 & 2
- `perf-ninja::loop_interchange` 1 & 2
- `perf-ninja::loop_tiling_1`

2. 描述您将采取哪些步骤来找出应用程序是否利用了所有利用 SIMD 代码的机会?
3. 尝试在实际代码上手动进行循环优化（但不要提交）。确保所有测试仍然通过。
4. 假设您正在处理一个 IpCall（每次调用指令）指标非常低的应用程序。您将尝试应用/强制哪些优化?
5. 每天运行您正在使用的应用程序。找到程序中最热的循环。它是向量化的吗? 可以强制编译器自动向量化吗? 附加问题：循环是由依赖链还是执行吞吐量导致瓶颈?

AI 回答：

1. **解决实验作业**：
   - 对于 `perf-ninja::function_inlining_1`，您需要检查函数调用是否可以被内联以减少调用开销，并分析内联对性能的影响。
   - 对于 `perf-ninja::vectorization` 1 & 2，您应该分析代码中的循环，看是否有可能通过向量化来并行化计算，以及如何通过编译器优化或手动优化来实现向量化。
   - 对于 `perf-ninja::dep_chains` 1 & 2，您需要识别并优化数据依赖链，可能通过循环变换或重新组织计算来减少依赖性。
   - 对于 `perf-ninja::compiler_intrinsics` 1 & 2，您应该使用编译器提供的内联函数（如 SIMD 指令）来手动优化代码，以提高性能。
   - 对于 `perf-ninja::loop_interchange` 1 & 2，您需要考虑循环交换，即改变循环的顺序，以改善内存访问模式和缓存利用率。
   - 对于 `perf-ninja::loop_tiling_1`，您应该实施循环平铺（Tiling），将大循环分解为更小的块，以提高缓存利用率和减少内存延迟。
2. **找出应用程序是否利用了所有 SIMD 代码的机会**：
   - 使用性能分析工具（如 Intel VTune Amplifier）来识别程序中的热点。
   - 查看编译器的优化报告，了解哪些循环已经被向量化。
   - 使用编译器的向量化指令（如 GCC 的`#pragma GCC vectorize`）来强制向量化。
   - 对于关键循环，手动重写代码以使用 SIMD 指令，并比较性能差异。
3. **手动进行循环优化**：
   - 选择一个实际的代码循环，例如一个计算密集型的循环。
   - 应用循环展开、循环取消开关、循环交换或循环平铺等技术。
   - 确保优化后的代码逻辑正确，并进行单元测试以验证功能。
   - 使用性能分析工具来比较优化前后的性能差异。
4. **处理 IpCall 指标非常低的应用程序**：
   - 分析程序中的函数调用，看是否有不必要的开销，考虑使用内联函数。
   - 识别并优化数据依赖链，减少循环中的依赖性。
   - 应用循环优化技术，如循环展开和循环平铺，以提高并行性和缓存利用率。
   - 使用编译器内联函数或手动编写 SIMD 代码来向量化关键循环。
5. **找到程序中最热的循环并分析**：
   - 使用性能分析工具来识别程序中的热点循环。
   - 检查这些循环是否已经被向量化，如果没有，尝试使用编译器优化或手动优化来实现向量化。
   - 分析循环的性能瓶颈，确定是由数据依赖链还是执行吞吐量引起的。如果是数据依赖链，尝试打破依赖链；如果是执行吞吐量，考虑使用更高效的指令或向量化。

章节总结

- 低效的计算是现实世界应用程序中瓶颈的重要部分。现代编译器通过执行许多不同的代码转换，非常擅长消除不必要的计算开销。尽管如此，我们仍然有很大的机会做得比编译器提供的更好。
- 在 Chapter 9 中，我们展示了如何通过强制某些代码优化来搜索程序中的性能提升空间。我们讨论了诸如函数内联、循环优化和向量化等流行的转换。